Chapter 2
System Configuration

The setup of the Tangible Landscape system consists of four primary components: (a) a physical model that can be modified by a user, (b) a 3D scanner, (c) a projector, and (d) a computer installed with GRASS GIS for geospatial modeling and additional software that connects all the components together. The physical model, placed on a table, is scanned by the 3D scanner mounted above. The scan is then imported into GRASS GIS, where a DEM is created. The DEM is then used to compute selected geospatial analyses. The resulting image or animation is projected directly onto the modified physical model so that the results are put into the context of the modifications to the model.

2.1 Hardware

Tangible Landscape can be built with affordable, commonly available hardware: a 3D scanner, a projector and a computer. We describe some of the current hardware options while noting that the technology develops rapidly and alternative, more effective solutions may emerge.

2.1.1 3D Scanner

In the Tangible Landscape workflow the 3D scanner captures the physical model as it is modified by users. Therefore it is important that the device scans as accurately as the technology allows. Few affordable 3D scanners are currently available. However, due to growing demand this technology is developing rapidly and new, improved 3D scanners may become available soon. The Tangible Landscape system was developed using the Kinect for Windows v2 and thus has the disadvantage of

© The Author(s) 2015

A. Petrasova et al., *Tangible Modeling with Open Source GIS*,
DOI 10.1007/978-3-319-25775-4_2

being bound to the Microsoft Windows operating system. Being bound to a specific operating system limits the choice of environment in which to run the application. Therefore, the selection of the 3D scanner may change in the future as new, more flexible technologies become available.

The Technology Behind 3D Scanning We describe the basic principles behind current 3D depth sensing technologies in order to explain both the potential and the limitations of current scanning devices. More detailed information can be found for example in Mutto et al. (2012).

Several currently available depth sensors such as Apple Primesense Carmine, Asus Xtion PRO LIVE,[1] and Kinect for Windows v1, are based on a Primesense proprietary light-coding technique. This technique uses triangulation to map 3D space in manner similar to how the human visual system senses depth from two slightly different images. Rather than using two cameras it triangulates between a near-infrared camera and a near-infrared laser source. Corresponding objects need to be identified in order to triangulate between the images. A light coding (or structured light) technique is used to identify the objects. The laser produces a pseudo-random dot pattern which is then used to find the matching dot pattern in the projected pattern. In this way the final depth image can be constructed.

Optical triangulation is also used by high resolution 3D scanners such as SCANIFY by Fuel3D (2015). The scanner acquires a series of stereoscopic photographs with different lighting directions. When scanning multi-color objects with some texture this technology can create highly accurate 3D models with high levels of detail. The major limitation is that only relatively small models that can be scanned making it unsuitable for most applications using physical landscape models. These models usually need to be relatively large to clearly represent relief at a landscape scale without too much vertical exaggeration.

Kinect for Windows v2 uses *time-of-flight*, a technique widely used in lidar technology. Kinect v2 has a sensor that indirectly measures the time it takes for pulses of near-infrared laser light to travel from a laser source, to a target surface, and then back to the image sensor. Time-of-flight sensors are generally considered to be more precise, but also more expensive.

Since the scanning and 3D modeling algorithms and their implementations for most sensors are proprietary, the specific behavior and precision of particular sensors is subject to many experimental studies. Generally, sensors using near-infrared light are sensitive to lighting conditions, so outdoor usage is typically not recommended. The depth resolution decreases with the distance from the sensor and also at close range (i.e. between a couple of millimeters to tens of centimeters). The range and field of view of the sensors can vary; Tangible Landscape requires short range sensors with a minimum distance of 0.5 m to keep the highest possible resolution. When scanning with one sensor the size of the physical model is limited by the required accuracy because the sensor must be far enough away to

[1] https://www.asus.com/us/Multimedia/Xtion_PRO_LIVE/

capture the entire model in the sensor's field of view, which for the Kinect for Windows v2 is 60° vertical by 70° horizontal. Detailed information about accuracy and precision of Kinect for Windows v1 and v2 can be found in Andersen et al. (2012); Lachat et al. (2015) and a comparison to Asus Xtion can be found in Gonzalez-Jorge et al. (2013).

2.1.2 Projector

The projector projects the background geospatial data and results of analyses onto the the 3D physical model. Therefore it is important to select a projector with sufficient resolution and properties that minimize distortion and generate a bright image.

Resolution and brightness are two important criteria to be considered. We recommend higher resolution projectors offering at least WXGA (1280 × 800). The brightness depends upon where Tangible Landscape is used and whether the room's ambient light can be reduced for the sake of brighter projected colors. We recommend brighter projectors (at least 3000 lumens) since we project on a variety of materials which are not always white and reflective.

There are two important types of projectors—standard and short-throw projectors. They differ in throw ratio values, which are defined as the distance from the projector's lens to the screen, divided by the width of the projected image (short throw and ultra short throw projectors have ratios 0.3–0.7, while the standard projectors have throw ratio values around 2). At the same distance the projector with a lower throw ratio can display a larger image. In other words a projector with a lower throw ratio (short-throw) can project an image of the same size as the higher throw ratio projector, but from a shorter distance. Tangible Landscape can be set up using both types of projectors; each has advantages and disadvantages.

The placement and configuration of the projector is important because it affects the coverage, distortion, and visibility of the projected data. For example in some setups the 3D scanner may be caught in the projection and would thus cast a shadow over the model. Therefore it can be practical to use a short-throw projector because it can be placed to the side of the physical model at a height similar to the 3D scanner (Figs. 2.1b and 2.2). Since the projection is cast from the side the projection beam does not cross the 3D scanning device and no shadow is cast. However, with a short-throw projector there is a certain level of visible distortion when projecting on a physical model that has substantial relief. The distortion occurs because the light rays reach the model at a very acute angle. The horizontal position at which the projected light intersects the model is shifted from the position at which it would intersect with a flat surface. Larger differences in height increase the distortion. Theoretically we can remove the distortion by either warping the projected data itself or using the projector to automatically warp the projected image. The first solution would require an undistorted dataset for geospatial modeling and a warped copy of that data for projecting. That is impractical especially when working with

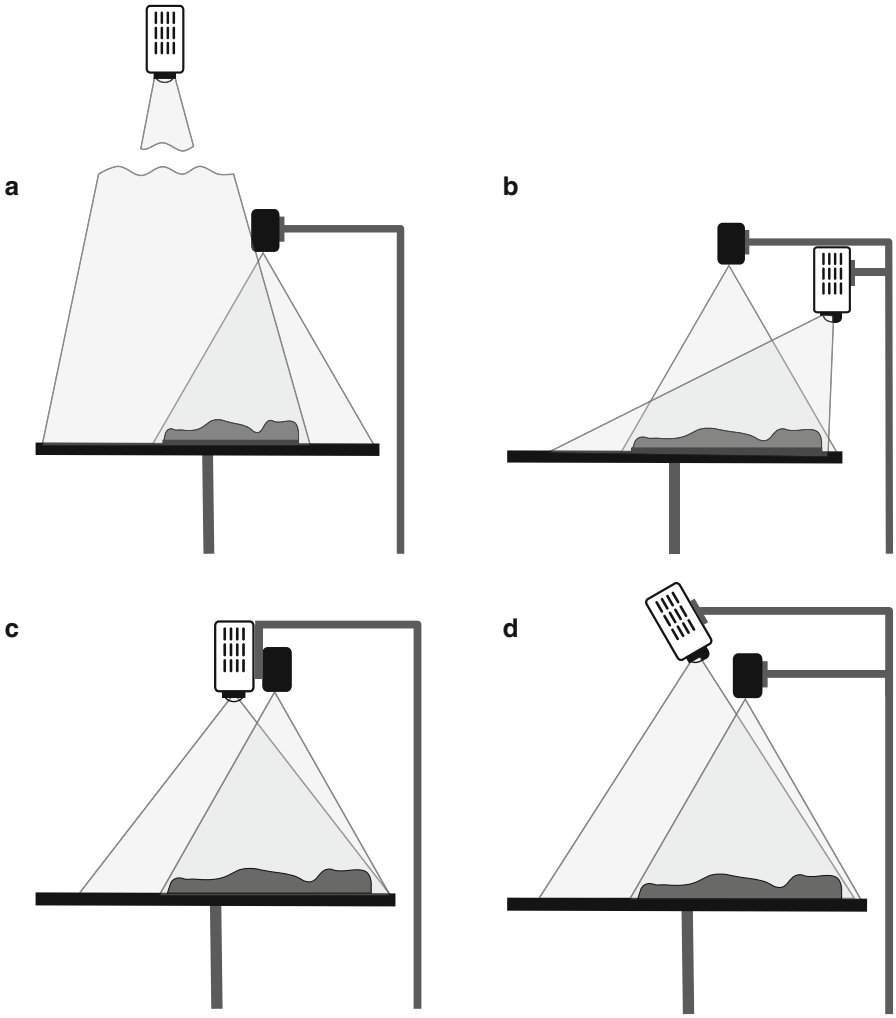

Fig. 2.1 Possible placements of the projector and scanner according to projector type. (**a**) Standard projector mounted in the ceiling. (**b**) Standard short throw projector. (**c**) Short throw projector capable of projecting above the center. (**d**) Short throw projector tilted to project above the center with keystone correction

many different raster and vector layers. The other solution requires the projector itself to warp the image; while this technology exists, it is only offered by a few projector manufacturers and such projectors are typically more expensive. Moreover, as the landscape is modified, the warping pattern should change as well. Currently it is not possible to find this feature in the off-the-shelf projectors.

With standard projectors the distortion is usually negligible since the incidence angle of the rays is relatively small. A standard projector needs to be placed much

Fig. 2.2 Mobile system setup: projector, scanner, model and laptop

higher above the model than a short-throw projector does due to the difference in throw ratios (Figs. 2.1a and 2.3). This creates several challenges. It is hard to mount and manipulate the projector when is it so high above the model. Furthermore it is challenging to align projector and the 3D scanner without casting a shadow. Since the 3D scanner has a limited field of view it must be placed close to the horizontal center of the model. When the projector is mounted above the scanner, the scanner is caught in the projector's beam, casting a shadow over part of the model. For small to mid sized models, depending on the particular setup and the height of the scanner and projector, this may not be a problem.

The shadow of the scanner can also be avoided with specialized short throw projectors that allow greater installation flexibility through lens shifting. These projectors (for example Canon WUX400ST) are capable of projecting from the horizontal center of the projected area at the same height as the scanner (Fig. 2.1c). A device which combines the scanner and projector would make this setup easy; an appropriate device, however, was not available at the time of writing.

To minimize the distortion when using projectors with lower throw ratios, we can project from the center by tilting the projector and correcting the resulting keystone distortion (Fig. 2.1d). With this setup we recommend projectors that have throw ratios around 1.0 (for example Epson PowerLite 1700 Series) as a lower ratio can result in additional distortion. When testing the projector setup it is useful to project the rectangular grid on a flat surface in order to quickly check if there is any distortion.

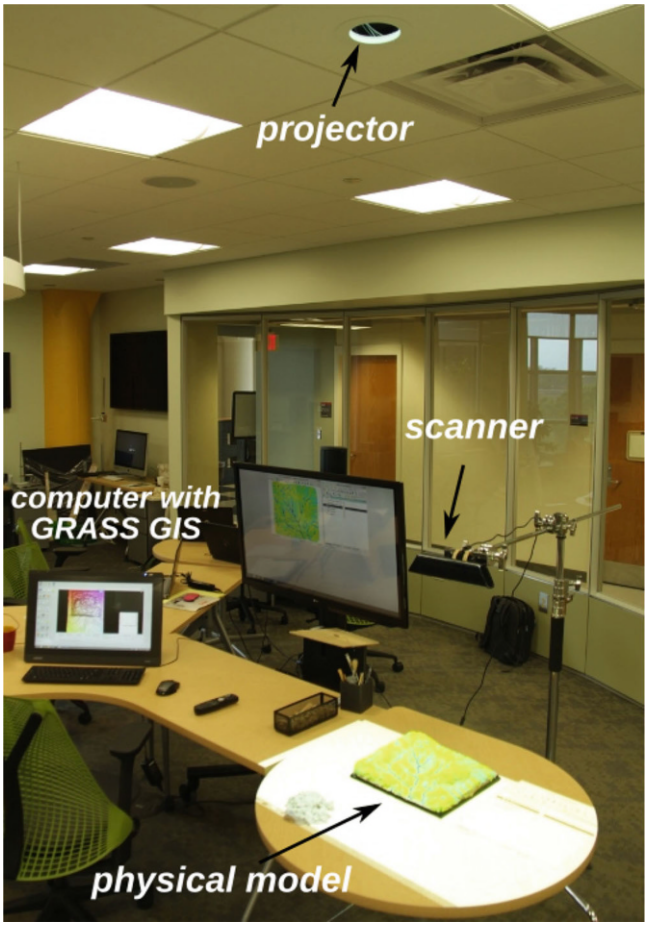

Fig. 2.3 Laboratory system setup with the projector mounted in the ceiling

2.1.3 Computer Requirements

System requirements depend largely upon the sensor and its associated library or software development kit (SDK). Certain sensors, such as Kinect for Windows, are designed to work only on specific operating systems. The computer should be configured for 3D scanning and geospatial modeling, both of which are performance- and memory-intensive processes. The hardware requirements are very similar to the requirements for gaming computers: a multi-core processor, at least 4 GB of system memory and a good graphics card are necessary to achieve real-time interaction with the model. The specific parameters required for the scanner device should be reviewed on the manufacturer's website.

2.1.4 Physical Setup

The setup for Tangible Landscape system is quite flexible. However, there are some crucial components and some specifics that make a setup more usable. When building Tangible Landscape in the laboratory or when bringing it to the community the following items are necessary:

- a table for a model
- a laptop or desktop computer with a table
- a scanner
- a projector
- 1–2 stands for a projector and a scanner
- 3–4 power plugs (and/or extension cable)

Ideally the table should be either a 90 cm × 90 cm table with rounded corners, a rounded table 100 cm in diameter, or a teardrop table of similar size. To freely interact with the model a person should be able to almost reach the other side of the table; this is not possible with larger tables. Smaller tables, on the other hand, have less space for tools, additional sand, and application windows. Ideally application windows with additional information should be projected onto the table beside the model. A large model can be placed on top of a smaller table or stand, provided that this base is stable. In this case if any application windows are needed, they have to be projected directly onto the model.

A round table is quite advantageous because people can stand at equal distances from the model and can easily move around the table. Unfortunately, one side of the table is typically occupied by the metal stands for projector and scanner since ceiling mounts are rarely possible and render the system immobile. The table should be stable enough to hold sand and models and sturdy enough to withstand their manipulation. We recommend putting the computer on a separate table so that the modeling table is not cluttered. The more space there is around the modeling table, the more access users have to the model. The computer, however, needs to be situated so that its operator has easy access to the model as well.

An alternative is to use a rectangular table (ideally with rounded corners) that is 140 cm × 90 cm for both the model and the laptop. A narrow table that is 150 cm × 75 cm may work as well. This setup makes the model less accessible as at least two sides of the model are blocked. However, the whole model should be accessible from any of the remaining sides, so this setup does not necessarily limit interaction, but rather the number of users.

Sometimes it is advantageous to have a large screen showing additional data or 3D views to enhance users' understanding of the processes. However, such screen should not limit access to the physical model. Tangible Landscape could be also extended with additional devices such as 3D displays, head-mounted displays, and hand-held devices.

There should be at least 80 cm of space on each side of the modeling table so that there is room for walking, standing, and placing the stands. The whole area required is about 2.5 m × 2.5 m but in practice the area is often larger depending on the size

of the tables and the surroundings. Generally, it is better to have a larger open space. With more space people can easily come and cluster around Tangible Landscape spreading around the different sides of the setup.

The projector and scanner can be mounted on one or two stands according to projector's capabilities as discussed in detail in Sect. 2.1.2. Two separate stands or arms give enough flexibility to accommodate various types of projectors. In any case, the stand for the projector must be robust because some of the projectors with adequate parameters can be heavy. However, there exist some portable projectors weighting less than 2 kg as well. A mobile setup with two separate stands and two separate tables is shown in Fig. 2.2. For permanent setups the projector can be mounted in the ceiling as shown in Fig. 2.3.

2.2 Software

The software for Tangible Landscape consists of several different components: a scanning program dependent upon the Kinect for Windows SDK libraries, GRASS GIS for processing scans and running geospatial analyses, and a custom library that automates the entire process. Here we briefly describe the software components with the main focus on GRASS GIS as the processing engine of Tangible Landscape. Tangible Landscape software is available in the GitHub repository[2] under the GNU GPL license.

2.2.1 GRASS GIS

GRASS GIS (Neteler and Mitasova 2008) is a general purpose, cross-platform, open-source geographic information system with raster, vector, 3D raster and image processing capabilities. It includes more than 400 modules for managing and analyzing geographical data and many more user contributed modules available in the add-on repository. GRASS GIS modules can be run using a command-line interface (CLI) or a native graphical user interface (GUI) called wxGUI which offers a seamless combination of GUI and CLI native to the operating system.

Modules are organized based on the data type they handle and they follow naming conventions explained in Table 2.1. Each module has a set of defined options and flags which are used to specify inputs, outputs, or different module settings. Most core modules using GRASS GIS C library are written in C for performance and portability reasons. Other modules and user scripts are written in Python using the GRASS GIS Python Scripting Library which provides a Python interface to GRASS GIS modules.

[2] https://github.com/ncsu-osgeorel/grass-tangible-landscape

Table 2.1 Naming conventions for GRASS GIS modules with examples

Name	Data type	Examples
g.*	General data management	g.list, g.remove, g.manual
r.*	Raster data	r.neighbors, r.viewshed, r.cost
r3.*	3D raster data	r3.colors, r3.to.rast, r3.cross.rast
v.*	Vector data	v.net, v.surf.rst, v.generalize
db.*	Attribute data	db.tables, db.select, db.dropcolumn
t.*	Temporal data	t.register, t.rast.aggregate, t.vect.extract
i.*	Imagery data	i.segment, i.maxlik, i.pca

GRASS GIS software can be downloaded freely from the main GRASS project website.[3] The download web page offers easy to install binary packages for GNU/Linux, Mac OS X, and Microsoft Windows as well as the source code. The GRASS GIS website also provides additional documentation including manual pages, tutorials, information about the externally developed modules (add-ons) and various publications. Support for developers and users is provided by several mailing lists. The following tutorial provides a quick introduction to GRASS GIS: http://grass.osgeo.org/grass70/manuals/helptext.html.

2.2.2 GRASS GIS Python API

GRASS GIS provides several Python application programming interfaces (API) to access different functionalities and accommodate many different use cases. Tangible Landscape uses the GRASS GIS Python Scripting Library to easily build new workflows by chaining together existing GRASS GIS modules. We will show an example how to use this library to automate tasks and build new functionality.

GRASS GIS modules are called with the following command syntax:

$$\underbrace{\texttt{r.colors}}_{\text{module name}}\quad \underbrace{\texttt{-e}}_{\text{flag}}\quad \underbrace{\texttt{raster}}_{\text{option name}}\texttt{=}\underbrace{\texttt{elev_state_500m}}_{\text{option value}}\quad \texttt{color=elevation}$$

This command assigns a predefined color ramp `elevation` to the raster map `elev_state_500m`. It can be executed in a GRASS GIS session from a terminal or from a GRASS GIS GUI command console. The same command looks very similar when written using the Python Scripting Library—it is just adjusted to Python syntax. To run it we have to be in a GRASS GIS session and must first import the necessary library.[4] We will use the function `run_command` to call individual modules:

[3] http://grass.osgeo.org

[4] All further Python code snippets assume the library is already imported to avoid code duplication.

```
import grass.script as gscript
gscript.run_command('r.colors', flags='e',
    raster='elev_state_500m', color='elevation')
```

Besides `run_command`, other important functions are `read_command` for reading the raw module output, `parse_command` for reading module output already parsed into a Python dictionary, and `write_command` for passing strings from standard input. All GRASS GIS modules can be called using these functions. However, for some commonly used modules the library conveniently provides wrappers simplifying the syntax, such as the `mapcalc` function, a wrapper for the *r.mapcalc* module.

The following code snippet provides a more complex example using these functions. Here, we extract low-lying areas from a DEM where the elevation z is lower than the difference $z_{mean} - z_{stddev}$:

```
import grass.script as gscript

def main():
   input_raster = 'elevation'
   output_raster = 'low_areas'
   stats = gscript.parse_command('r.univar', map=input_raster,
       flags='g')
   mean = float(stats['mean'])
   stddev = float(stats['stddev'])
   low = mean - stddev
   gscript.mapcalc('{out} = {inp} <
       {lim}'.format(out=output_raster, inp=input_raster,
       lim=low))

if __name__ == "__main__":
   main()
```

More information on writing Python scripts in GRASS GIS can be found in the online GRASS GIS Python Scripting Library documentation. [5]

2.2.3 Scanning Component

The scanning component is currently a C++ application derived from the sample application KinectFusionExplorer shipped with the Kinect for Windows SDK. It reconstructs a mesh based on multiple scans using the KinectFusion algorithm (Newcombe et al. 2011) and writes the point cloud as a text file. It allows us to set the scanning properties such as the resolution of the reconstructed mesh and to limit the volume of the mesh. We recommend using a higher volume resolution such

[5] GRASS GIS Python Scripting Library documentation: http://grass.osgeo.org/grass70/manuals/libpython/script_intro.html

Fig. 2.4 A mesh with a resolution of 512 voxels/m constructed using the KinectFusion algorithm. The mesh captures a sand model, the area outside of the model including a pile of sand and a modeling tool, and the hands of a person interacting with the model

as 512 or 640 voxels/m. It is important to make the volume of the mesh as small as possible to increase processing speed and decrease memory demands. Since the size of the mesh can only be controlled in predefined steps we cannot limit the scanned area to the model area in this way—further processing of the point cloud to filter the points outside of the model area is needed (Fig. 2.4).

2.2.4 Tangible Landscape Plugin for GRASS GIS

The Tangible Landscape plugin connects the scanning component with GRASS GIS and automates the loop of scanning, importing scans, and geoprocessing in the GRASS GIS environment. It has a graphical user interface which allows the adjustment of the different processing parameters that are necessary to properly georeference and extract the model. First it can perform an initial calibration. Then with each new scan the plugin performs the following basic steps:

1. acquiring the scan as a point cloud,
2. extracting only relevant points from the point cloud (filtering points outside the model area)
3. georeferencing (horizontal rotation, horizontal and vertical scaling, translation) the scanned data to known geographic coordinates on the edges of the model,
4. importing the point cloud into GRASS GIS and creating a digital elevation model,
5. running selected geospatial analyses, and
6. re-rendering the resulting raster or vector data that are projected on top of the model.

The conversion from the scanned point cloud to a DEM is an automated process; the details and assumptions that have to be met to produce good results are described in the following paragraphs.

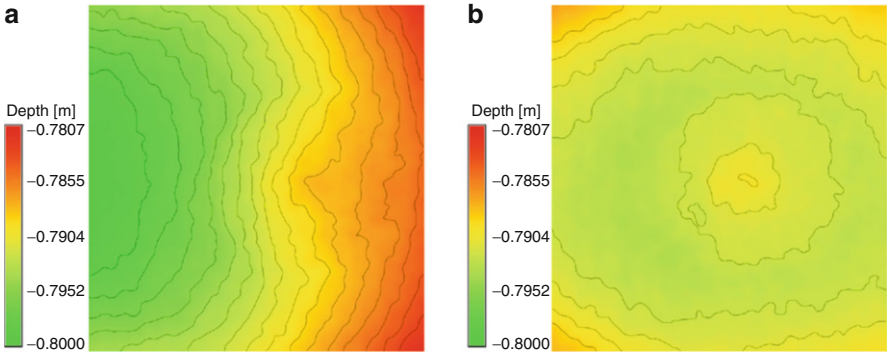

Fig. 2.5 Even slight angular deviation from the vertical scanner axis will cause a tilted scan as seen in (**a**) on a scan of a flat plane with a small angular deviation of 1.7°. A calibration based on the initial scan of a flat plane can compensate for such angular deviations resulting in a horizontally aligned scan as shown in (**b**) although a small radial distortion will still be present

Calibration During scanning the scanner axis should be oriented precisely perpendicular to the table with the model in order to avoid a tilted scan. Even a small deviation of 1° can cause centimeters of height difference depending on the size of the model (see Fig. 2.5a). Therefore, we first calibrate the system by scanning the empty table and computing the angular deviation. We then use this information to automatically rotate each scanned point cloud.

From the scan of the empty table, we compute the plane that best fits the scan by applying the linear regression in 3D. We compute the angular deviation between the unit vector along the scanner axis and the vector perpendicular to the plane. We then use Rodrigues' rotation formula (Wikipedia 2015) to derive a rotation matrix to rotate the point cloud by this angular deviation around a vector given as a cross product of the two vectors. After calibration, the derived rotation matrix is used for rotating every scan resulting in perfectly horizontal scan, see Fig. 2.5b. If the scanner is moved the calibration must be repeated.

For more precise results, we can also minimize the radial distortion of the scanner visible in Fig. 2.5b. The simplest way to do this is to acquire a scan of a flat surface with the dimensions of the physical model and then subtract it from the raster representation of the model.

Model Extraction Since the size of the scanned area cannot be easily limited to just the physical model (see Fig. 2.4), we need to find the boundaries of the model in order to georeference the scanned data.

The first part of the process is to identify points that represent the underlying table. These points have very similar (but due to noise nonidentical) low values of z coordinates. If enough of the table is captured in the scan, we can find the interval of z coordinates containing the points of the table as a first local maximum in a histogram of z values. These points can then be discarded.

After removing these points, we attempt to remove points outside of the model area such as leftover sand or arms reaching over the model in Fig. 2.4. This can be successfully done when the model is rectangular, since we can estimate the horizontal dimensions of the model from the scan and then trim the areas that do not contain enough of points from each side. If the model is not rectangular we can manually choose how many centimeters to trim from each side. Manual trimming, however, requires adjustments anytime that the model is moved even slightly.

Georeferencing Georeferencing the scanned model is an important step when we need to combine it with our geographic data and to ensure that any geospatial analyses are performed on a DEM with real-world dimensions. We need to designate the DEM raster map that the model represents and specify the vertical exaggeration of the physical model to scale the elevation values properly. The scale factors S_x, S_y and S_z are computed:

$$S_x = \frac{Y_{north} - Y_{south}}{y_{max} - y_{min}}, \quad S_y = \frac{X_{north} - X_{south}}{x_{max} - x_{min}}, \quad S_z = \frac{(S_x + S_y)/2}{e} \tag{2.1}$$

where X, Y are DEM (real-world) coordinates, x, y are coordinates on the physical model and e is the specified vertical exaggeration. The vertical scale of the model typically differs from the horizontal because we vertically exaggerate the physical model to enhance our perception of the landscape and simplify our interaction with the model. Since Tangible Landscape users usually interact with the physical model from the side opposite to the scanner we have to rotate the scan by 180°. We can georeference the point cloud with the following equation:

$$\begin{bmatrix} X \\ Y \\ Z \end{bmatrix} = \begin{bmatrix} S_x & 0 & 0 \\ 0 & S_y & 0 \\ 0 & 0 & S_z \end{bmatrix} \cdot \begin{bmatrix} \cos\alpha & -\sin\alpha & 0 \\ \sin\alpha & \cos\alpha & 0 \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} x \\ y \\ z \end{bmatrix} + \begin{bmatrix} t_x \\ t_y \\ t_z \end{bmatrix} \tag{2.2}$$

which rotates the points around the z axes by angle α in the counterclockwise direction, scales to real-world dimensions, and translates the points by adding t_x and t_y computed so that the lower left corner of the model matches the south-west corner of the DEM. The vertical translation t_z is then similarly computed to match the lowest point of the model and the minimum height of the DEM.

DEM Processing The plugin allows users to select the method for creating the DEM from the point cloud, which can be either binning or interpolation (see Sect. 4.1). Once the DEM is created all geospatial analyses specified in a dedicated Python file are sequentially performed. Each separate analysis is represented as one function. These functions are reloaded for each new processing loop so that we can easily run different algorithms and change their parameters. This flexible system enables us to dynamically develop and test new workflows during scanning. The plugin provides a library containing often used analyses and workflows which can be readily used and combined with other GRASS GIS functionality using the GRASS GIS Python Scripting Library.

References

Andersen, M., Jensen, T., Lisouski, P., Mortensen, A., Hansen, M., Gregersen, T., and Ahrendt, P. (2012). Kinect Depth Sensor Evaluation for Computer Vision Applications. Technical report, Aarhus University, Department of Engineering. Denmark.

Fuel3D. SCANIFY: The low-cost, high-quality handheld 3D Scanner from Fuel3D [online]. (2015) [Accessed 2015-08-13]. https://www.fuel-3d.com/.

Gonzalez-Jorge, H., Riveiro, B., Vazquez-Fernandez, E., Martínez-Sánchez, J., and Arias, P. (2013). Metrological evaluation of Microsoft Kinect and Asus Xtion sensors. *Measurement*, 46(6):1800–1806.

Lachat, E., Macher, H., Mittet, M.-a., Landes, T., and Grussenmeyer, P. (2015). First Experiences With Kinect V2 Sensor for Close Range 3D Modelling. *ISPRS - International Archives of the Photogrammetry, Remote Sensing and Spatial Information Sciences*, XL-5/W4(February):93–100.

Mutto, C., Zanuttigh, P., and Cortelazzo, G. (2012). Introduction. In *Time-of-Flight Cameras and Microsoft Kinect^{TM}*, SpringerBriefs in Electrical and Computer Engineering, pages 1–15. Springer US.

Neteler, M. and Mitasova, H. (2008). *Open source GIS: a GRASS GIS approach*. New York: Springer, third edition.

Newcombe, R. a., Molyneaux, D., Kim, D., Davison, A. J., Shotton, J., Hodges, S., and Fitzgibbon, A. (2011). KinectFusion : Real-Time Dense Surface Mapping and Tracking. *IEEE International Symposium on Mixed and Augmented Reality*, pages 127–136.

Wikipedia. Rodrigues' rotation formula — wikipedia, the free encyclopedia [online]. (2015) [Accessed 2015-08-13]. https://en.wikipedia.org/w/index.php?title=Rodrigues%27_rotation_formula&oldid=671556479.